

Midterm Examination Problem Sheet

TIME: 04/18/2023, 13:20–16:10

This is an open-book exam. You can use any printed materials as your reference during the exam. Any electronic devices are not allowed.

Any form of cheating, lying or plagiarism will not be tolerated. Students can get zero scores and/or get negative scores and/or fail the class and/or be kicked out of school and/or receive other punishments for those kinds of misconducts.

Both English and Chinese (if suited) are allowed for answering the questions. We do not accept any other languages.

*Unless explicitly asked, you do **not** need to justify why your algorithm runs in the desired time/space complexity, but you are welcomed to briefly explain the idea to help the TAs grade your solution.*

There are 16 problems. 8 of them are marked with * and are supposedly simple(r); 4 of them are marked with ** and are supposedly regular; 4 of them are marked with *** and are supposedly difficult. The simpler ones are worth 10 points, and others are worth 15 points. The full credit is 200 points. The 16 problems are organized to 8 sections, each with two problems. You can use one and only one A4 paper (one-side only) per section. For each section, please write down the section ID (top-left), your name and school ID (top-right) on the A4 paper. Then, write down the problem ID and your answer to each problem as instructed. You should keep your answers as *concise* as possible to facilitate grading. In particular, please use proper *pseudo code* and/or clear English/Chinese descriptions to illustrate your algorithm!

A Arrays

1. (*) Consider an array `arr` of length n that contains different non-zero integers. Design an $O(n)$ -time and $O(1)$ -extra-space algorithm that moves the positive integers to the beginning of the array and the negative integers to the end of the array. That is, the algorithm should change the original array to a modified array with exactly the same set of elements, while returning some k such that `arr[i]` is positive for all $1 \leq i \leq k$ and `arr[i]` is negative for all $k < i \leq n$. The order within the positive elements and the order within the negative ones can be arbitrary.
2. (***) Consider a researcher with n published documents, where the i -th paper is cited a_i times. Furthermore, assume that $a_1 \geq a_2 \geq a_3 \geq \dots \geq a_n$. The H-index of the researcher is computed as the maximum k such that $a_k \geq k$. For instance, Google Scholar shows that Professor Hsuan-Tien Lin has 109 published documents. The highest cited one is of $a_1 = 1226$, followed by $a_2 = 1072$, $a_3 = 936$, $\dots$, $a_{32} = 34$, $a_{33} = 32$, $\dots$, $a_{107} = a_{108} = a_{109} = 0$. That is, the H-index of Professor Lin is 32, meaning that he has published 32 documents, each of which has been cited at least 32 times. Design an $O(\log n)$ -time and $O(1)$ -extra-space algorithm that helps any scholar calculate his H-index when given an array of non-increasing elements $a_1, a_2, \dots, a_n$.

B Linked Lists

3. (*) Consider a length- $2K$ singly linked list like this

$$\text{head} \rightarrow a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_K \rightarrow a_{K+1} \rightarrow \dots \rightarrow a_{2K} \rightarrow \text{NIL}$$

with a given $K \geq 1$. The linked list contains integers $a_1, a_2, \dots, a_{2K}$. Design an $O(K)$ -time and $O(1)$ extra-space algorithm that folds the linked list, which means that the resulting singly linked list should be of length K and look like

$$\text{head} \rightarrow (a_K + a_{K+1}) \rightarrow (a_{K-1} + a_{K+2}) \rightarrow \dots \rightarrow (a_1 + a_{2K}) \rightarrow \text{NIL}$$

You are allowed to modify the **data** and **next** fields of any given node, as well as freeing any existing node to the system. But you are *not* allowed to create any new nodes.

4. (**) Run-length encoding is a very common way of data compression. Consider an array

$$a = [0, 1, 1, 1, 3, 3, 3, 3, 5, 5, 6, 6, 1].$$

Its run-length encoding can be viewed as a linked list of pairs (**value**, **count**) like

$$(1, 0) \rightarrow (3, 1) \rightarrow (4, 3) \rightarrow (2, 5) \rightarrow (2, 6) \rightarrow (1, 1) \rightarrow \text{NIL}$$

which means there is one “0”, followed by three “1”, followed by four “3”, followed by two “5”, followed by two “6”, followed by one “1”. Given the run-length encoding of two length- n arrays a and b as linked lists L_a with n_a nodes and L_b with n_b nodes. With the definition of run-length encoding, we can easily see that $n_a \leq n$ and $n_b \leq n$. Design an $O(n_a + n_b)$ -time and $O(1)$ -extra-space algorithm that computes $\sum_{i=1}^n (a[i] - b[i])^2$.

C Stacks, Queues, and Deques

5. (*) Considering representing a deque by a circular array of size 4, with an empty deque in the beginning. After executing each step of the following operations sequentially, list the contents of the circular array, with highlights on where the head and tail are.

```
pushFront(9); pushFront(3); pushBack(2);  
pushFront(5); popFront(); popBack();  
pushBack(7); popBack(); popFront(); pushBack(6)
```

6. (*) Consider an array `arr` of length n that contains integers. The next-larger element of `arr[i]` is defined as the smallest $j > i$ such that `arr[j] > arr[i]`. For instance, if

`arr = [5, 3, 4, 4, 8],`

the array of its next-larger elements are

`[8, 4, 8, 8, NIL].`

Design an $O(n)$ -time and $O(n)$ -extra space algorithm that *modifies* a given `arr` to the array of its next-larger elements.

D Trees and Binary Search Trees

7. (*) Describe a sequence of inserting 4 *different* integers into a binary search tree that results in the tallest possible binary search tree of 4 nodes. The sequence should be *neither* increasing *nor* decreasing. Then, draw the resulting binary search tree.

8. (***) For a general tree, its preorder traversal visits the root first, and then performs a pre-order traversal on each sub-tree.

PREORDER-TRAVERSAL(T)

```
1  if  $T$  is not NIL
2      action with  $T.data$ 
3      for  $i = 1$  to  $T.NumberOfSubTrees$ 
4          PREORDER-TRAVERSAL( $T.SubTree[i]$ )
```

Prove or disprove that the pre-order traversal on the “left-child right-sibling” representation of T (which is a binary tree) visits the nodes in the same order as the pre-order traversal on T .

E Heaps

9. (*) Consider a complete binary tree of integers stored in an array `arr` of length n . Design an $O(n)$ -time and $O(1)$ -extra-space algorithm that detects whether the complete binary tree is actually a heap or not.

10. (**) For a max-heap stored in an array `arr` of length $2^k - 1$ that contains distinct keys, where `arr[1]` is the root. What is the maximum number of elements that are bigger than `arr[4]`? Explain how you derive this number.

F Sorting

11. (*) The following implementation of the INSERTION-SORT algorithm is correct but possibly *not* stable.

```
INSERTION-SORT( $A$ )
1  for  $m = 2$  to  $A.length$ 
2       $key = A[m].key$ 
3       $i = m - 1$ 
4      while  $i > 0$  and  $A[i].key \geq key$ 
5           $A[i + 1] = A[i]$ 
6           $i = i - 1$ 
7       $A[i + 1] = A[m]$ 
```

Modify the algorithm to a stable version, and explain why your modification is effective. If you believe the algorithm to already be stable (really?), please explain why it is stable.

12. (**) Consider the following sorting algorithms: Selection Sort, Insertion Sort, Heap Sort, and Merge Sort. When the algorithm is fed with an array with a large length n that is already sorted, which algorithm runs the fastest? Explain your choice.

G Complexity!

For the next two sections, please prove the results using the following definition of the asymptotic notations (which is exactly the same as the notation you learned in our class).

For your proof, you can use

1. any results that have been proved in our class—please be extra careful on this. Our experience is that students sometimes “think” some results have been proven in the class, but actually, those results have not been proven. So it is very important to understand what have been proven and what have not been proven.
2. any results in related sections of your textbook, regardless of whether they have been taught.
3. any results in fundamental Calculus or Discrete Math classes.

Please briefly state where those results are from (to your belief) if you choose to use them.

Suppose that f and g are non-negative functions defined on $\mathbb{N}$ (if you need, you can also assume them to be defined on $\mathbb{R}$). We call $f(n) = O(g(n))$ if and only if there exist positive numbers $c \in \mathbb{R}^+$ and $n_0 \in \mathbb{N}$ such that

$$f(n) \leq cg(n) \text{ for all } n \geq n_0.$$

13. (*) Let $f(n) = n^a$ and $g(n) = n^b$ for $a \geq 0$ and $b \geq 0$. If $b \geq a$, prove that $f(n) = O(g(n))$.

14. (**) Let $f(n) = n^a$ and $g(n) = n^b$ for $a \geq 0$ and $b \geq 0$. If $f(n) = O(g(n))$, prove that $b \geq a$.

H Endless Complexity Ends the Midterm Exam!

15. (***) Assume that $f(n) = O(n^k)$ for some $0 < 1 \leq k$. Prove or disprove that $2^{f(n)} = O(2^n)$.
16. (***) Assume that $2^{f(n)} = O(2^n)$. Prove or disprove that $f(n) = O(n^k)$ for some $0 < 1 \leq k$. b

Congratulations on conquering the exam and thank you for a wonderful half-semester together!

Hsuan-Tien, 04/18/2023